



The University of Manchester

Retrieval-Augmented Generation (RAG)

Chenghua Lin

Department of Computer Science

The University of Manchester

Limitations of LLMs

- LLMs are typically trained with next token prediction
 - Learn statistical patterns from huge amount of text
 - Knowledge is stored implicitly in model parameters
- Limitations
 - Hallucinations: fluent but unsupported / incorrect outputs
 - Knowledge cutoff: limited to information seen during training
 - No access to private or up-to-date corpora

What is RAG?

- Retrieval-Augmented Generation (RAG) is a system architecture that:
 - **Retrieves** relevant passages from a background corpus
 - **Augments** the LLM input with retrieved passages as context
 - **Generates** an answer grounded in those retrieved evidences
- Key points
 - RAG does **not** update model weights
 - It augments the input context, not model parameters
 - Retrieval and generation are separate components but integrated at inference time

RAG = Information Retrieval + Language Model Generation

Why RAG?

- Explainability
 - LLM's outputs are conditioned on retrieved evidence rather than relying solely on internal parametric knowledge
- Privacy
 - Use proprietary data as external knowledge instead of incorporating it into training data
- Decomposition of Memorisation and Reasoning
 - The retriever handles knowledge access, allowing the LLM to focus on reasoning and generation
 - Small LLM + Retriever can outperform a larger standalone LLM in domain-specific settings

Architecture

- **Ingestion**

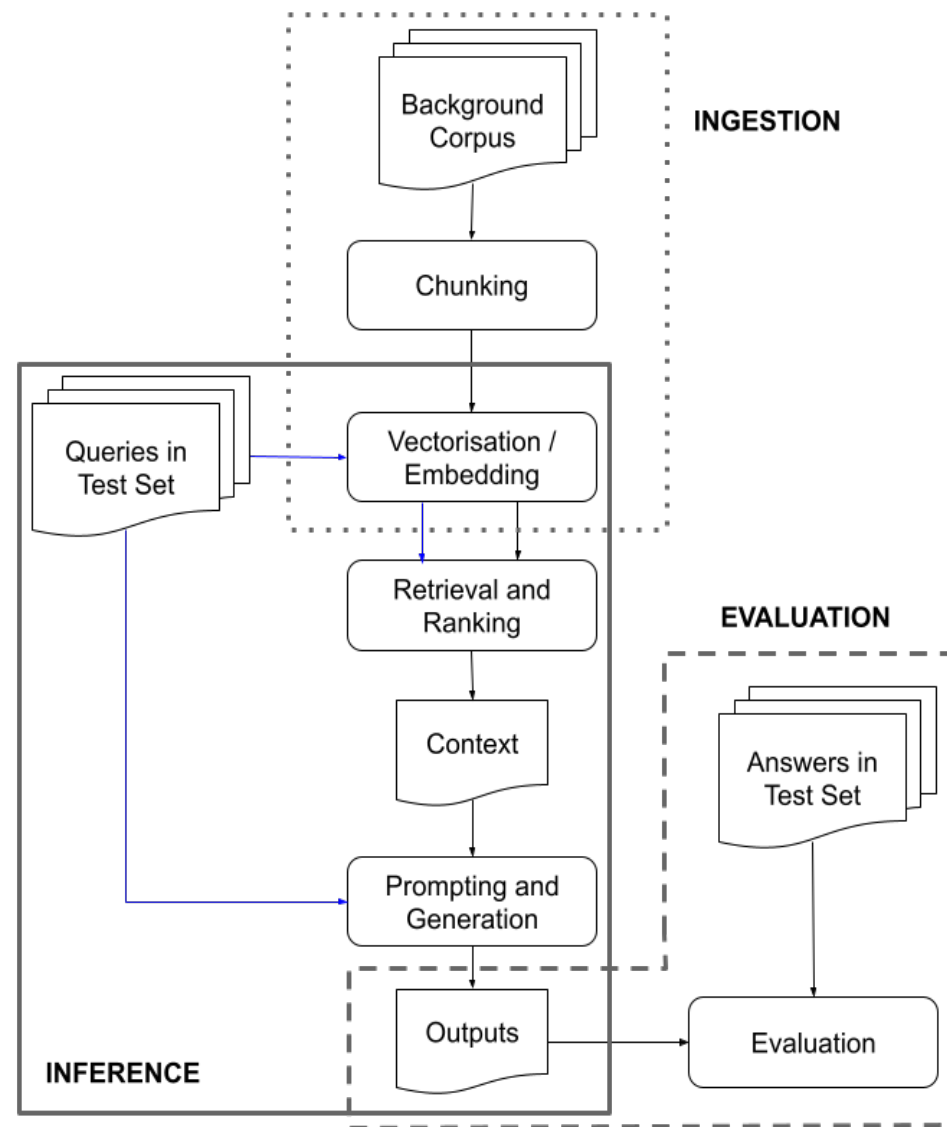
Corpus → Chunk → Embed →
Store in vector index

- **Inference**

Query → Embed → Retrieve →
Prompting → Generate

- **Evaluation**

- Retrieval quality (e.g., Recall@k)
- End-to-end answer quality (e.g., BERTScore)



Ingestion Pipeline - Chunking

- Why chunking?
 - LLMs have **context length limits**
 - Retrieval performs better when chunks preserve **coherent semantic units**
- Common chunking strategies:
 - Fixed-length token windows (e.g. 200-500 tokens per chunk)
 - Overlapping windows (e.g. 10-20% overlap)
 - Sentence-based chunking (split by sentence; merge into size-controlled chunks)
 - Structure-aware chunking (split by headings or sections)

Chunking: Fixed Length

Miso soup is a traditional Japanese soup. It is made from dashi stock and miso paste. Common additions include tofu and wakame. Dashi is typically made from kombu and katsuobushi.

- **Fixed-Length Windows:** (i) split by token count; (ii) may ignore sentence boundaries.
 - **Chunk 1:** Miso soup is a traditional Japanese soup. It is made from dashi stock
 - **Chunk 2:** and miso paste. Common additions include tofu and wakame.
- May break sentences → fragment meaning

Chunking: Overlapping Windows

Miso soup is a traditional Japanese soup. It is made from dashi stock and miso paste. Common additions include tofu and wakame. Dashi is typically made from kombu and katsuobushi.

- Fixed-size chunks with partial overlap (e.g., 300 tokens with 50-token overlap)
 - Chunk 1: It is made from dashi stock and miso paste.
 - Chunk 2: from dashi stock and miso paste. Common additions include tofu
- Reduces boundary loss; important facts near boundaries are less likely to be missed

Sentence-Based Chunking

Miso soup is a traditional Japanese soup. It is made from dashi stock and miso paste. Common additions include tofu and wakame. Dashi is typically made from kombu and katsuobushi.

- Split at sentence boundaries and merge into size-controlled chunks.
 - Chunk 1: Miso soup is a traditional Japanese soup.
 - Chunk 2: It is made from dashi stock and miso paste.
 - **Could be multiple sentences in a chunk**
- Preserves semantic units
- Improves readability of retrieved context

Chunking Trade-offs

- Larger chunks
 - More context per retrieval hit
 - Lower retrieval precision (more irrelevant text included)
 - Fewer chunks overall (coarser granularity)
- Smaller chunks
 - Higher retrieval precision
 - Finer granularity for matching
 - Risk of losing needed context (“semantic fragmentation”)
- Chunking affects retrieval metrics (Recall@k, MRR) and generated answer quality.

Embeddings in RAG

- **Embeddings:** dense vectors representing semantic meaning
- **How embeddings are used in RAG**
 - Offline: embed each chunk in the corpus
 - Online: embed the user query
 - Retrieve top-k similar chunks (e.g., cosine similarity)
- **Example embedding models**
 - all-MiniLM-L6-v2, E5-small, BGE-small

Embedding quality strongly impacts retrieval effectiveness

Retrieval in RAG (Bi-Encoder)

- Dense Retrieval (Bi-Encoder)
 - Precompute embeddings for all chunks (offline)
 - Embed user query at inference time (online)
 - Compute similarity in embedding space (e.g., cosine similarity)
 - Retrieve top-k most similar chunks
- Pros
 - Efficient and scalable
 - Strong recalls
- Cons
 - Limited fine-grained query–document interaction
 - May retrieve semantically similar but irrelevant chunks

Re-ranking in RAG

- Stage 1: Bi-Encoder (Dense Retrieval)
 - Encode query and documents independently
 - Retrieve top-N candidates using vector similarity
 - High recall, efficient
- Stage 2: Cross-Encoder (Re-ranker)
 - Jointly encode (query + document)
 - Compute a refined relevance score
 - Re-rank top-N → select final top-k
- Trade-Off
 - Re-ranking improves precision and ranking quality
 - More computationally expensive (applied only to a small candidate set)

Vector Store

- Why a vector store?
 - Efficient nearest-neighbour search over embedding vectors
 - Enables scalable retrieval over large embedding collections
- How it works
 - Stores chunk embeddings in an index
 - Uses Approximate Nearest Neighbour (ANN) search
 - Returns top-k most similar chunks
- ANN trades exact search for significant speed gains
 - Example systems: FAISS, Chroma, LanceDB

Exact Nearest Neighbour

- Exact Nearest Neighbour (NN)
- Given: N chunk embeddings; Query embedding q
- Exact search:
 - Compute similarity between q and every embedding
 - Return the true top- k most similar vectors
- Computational Cost
 - 1,000,000 vectors $\rightarrow$ 1,000,000 similarity computations per query
 - Too slow for large-scale RAG systems

Approximate Nearest Neighbour (ANN)

- Do **not** search the entire vector space.
- Instead:
 - Use an index structure to narrow the search
 - Explore only a small, promising region
 - Return vectors that are very likely nearest neighbours (not guaranteed exact)

Approximate Nearest Neighbour (ANN)

- Partition-Based Methods (e.g., IVF)
- Step 1 (offline)
 - Divide the vector space into clusters (e.g., using k-means)
 - Assign each embedding to a cluster
- Step 2 (at query time)
 - Identify the nearest cluster(s) to the query vector
 - Search only within those clusters
 - Dramatically reduces the number of similarity comparisons
 - Typical speed-up: 10x–100x; High recall in practice

Prompting in RAG

- Prompting: the bridge between retrieval and generation
- The prompt determines how retrieved evidence influences the model's output.
- Standard RAG Prompt Structure
 - Instruction: define behavior and constraints (system role)
 - Context: retrieved passages (+ metadata, chunk IDs)
 - Question: user task
 - Answer: model output

Standard RAG Prompt Structure

⌞⌟ Plain text

System:

You are a helpful assistant. Use ONLY the provided context.

Context:

[Chunk 1]

[Chunk 2]

...

User Question:

{question}

Answer:

Include chunk IDs / sources to enable attribution

Example: Strong Grounding Prompt

⟨⟩ Plain text

Use ONLY the information in the context.

If the answer is not in the context, say "Not found in provided documents."

Cite the chunk number for each claim.

- Why this matters:
 - Reduces hallucination
 - Enables explicit attribution (claim → evidence mapping)
 - Makes it easier to audit and automate faithfulness checks
 - Encourages evidence-constrained generation

Context Formatting

- Context Formatting Matters
 - Metadata helps reasoning
 - Chunk IDs enable citation
 - Clear structure reduces confusion in long contexts
 - Order chunks by retrieval score (highest first).
- Practical formatting patterns
 - Separate chunks with delimiters (e.g., ---)
 - Add (Source, ChunkID) headers per chunk

Format Examples

Miso soup is a traditional Japanese soup made from dashi stock and miso paste. Dashi is typically made from kombu and katsuobushi. Common additions include tofu and wakame.

[Chunk 1 | Source: FoodGuide.pdf | Section: Japanese Cuisine | Score: 0.92]
Miso soup is a traditional Japanese soup made from dashi stock and miso paste.

[Chunk 2 | Source: IngredientsGuide.pdf | Page: 14 | Score: 0.88]
Dashi is typically made from kombu and katsuobushi.

[Chunk 3 | Source: FoodGuide.pdf | Section: Additions | Score: 0.81]
Common additions include tofu and wakame.

Generation using LLMs

- LLMs generate text token-by-token conditioned on the prompt
- In RAG, retrieved context changes the conditional distribution:

$$P(\text{answer} \mid \text{question, retrieved context})$$

- RAG modifies the input distribution
- It does NOT modify model weights
- It shifts probability mass toward grounded tokens

Evaluation in RAG

We can evaluate a RAG system at two levels:

- **Retrieval quality:** Did the system retrieve the correct supporting evidence?
- **End-to-end QA quality:** Is the final generated answer correct compared to the gold answer?

If relevant evidence is not retrieved, the generator cannot produce a correct grounded answer.

Retrieval Evaluation: Recall@k

- **Recall@k**: Proportion of queries where at least one relevant chunk appears in the top-k results.
 - Measures **coverage** of relevant evidence
 - Ignores ranking order within top-k

$$\text{Recall@k} = \frac{\text{\#queries with } \geq 1 \text{ relevant chunk in top-k}}{\text{\#queries}}$$

- Recall@k → Did we retrieve relevant evidence?

Retrieval Evaluation: MRR

- **Mean Reciprocal Rank (MRR)**

- measures how early the first relevant chunk appears.

$$RR = \frac{1}{\text{rank of first relevant chunk}}$$

MRR = average RR over all queries

- Higher when relevant evidence appears earlier
- Sensitive to ranking quality

Retrieval Evaluation: nDCG

- **Normalized Discounted Cumulative Gain (nDCG)**

- Used when relevance is graded (not just relevant / not relevant).

$$\text{DCG} = \sum_{i=1}^k \frac{\text{rel}_i}{\log_2(i+1)} \qquad \text{nDCG} = \frac{\text{DCG}}{\text{ideal DCG}}$$

- k : number of retrieved results considered (e.g., top-10)
- i : rank position (1 = highest rank)
- rel_i : relevance score of the document at rank i
- Ideal DCG = DCG score of the best possible ranking
- nDCG normalizes the score to the range [0, 1].

Retrieval Evaluation: nDCG

- **Normalized Discounted Cumulative Gain (nDCG)**

- Used when relevance is graded (not just relevant / not relevant).

$$\text{DCG} = \sum_{i=1}^k \frac{\text{rel}_i}{\log_2(i+1)} \qquad \text{nDCG} = \frac{\text{DCG}}{\text{ideal DCG}}$$

- Rewards highly relevant documents ranked higher
 - Penalizes relevant documents ranked lower
 - Captures overall ranking quality
- Interpretation
 - High nDCG → relevant documents ranked near the top
 - Low nDCG → relevant documents ranked lower or poorly ordered

Metric Comparision

- Recall@k → Did we retrieve relevant evidence?
- MRR → How early does relevant evidence appear?
- nDCG → How well are all relevant results ranked?

End to End QA Quality

- End-to-End Answer Quality
 - Compare generated answer $\hat{y}$ vs. gold answer y
- Common metrics
 - Exact Match (EM)
 - Token-level F1
 - BERTScore
 - What else?
- Note: high overlap does not guarantee factual grounding

The Old Reliables: BLEU/ROUGE Score

Reference: I am giving a talk at a data science conference

Hyp 1: I am giving a talk at a conference about data science

lots of overlap → high score

Hyp 2: This talk is about recent advances in medical imaging

little overlap → low score

Why is Evaluation Hard?

Reference: I am giving a talk at a data science conference

Hyp 1: I am giving a talk at a political science conference

lots of overlap but bad output

Hyp 2: My lecture will be given to the meeting on data analytics

little overlap but good output
(particularly difficult for open-ended problems)

End-to-End Answer Quality

- Exact Match (EM)
 - Strict string-level equality
 - Common in extractive QA
 - Does **not** allow paraphrasing
- Token-Level F1
 - Based on token overlap between generated and gold answers
 - More forgiving than EM
 - Ignores word order but sensitive to lexical differences

$$F1 = \frac{2PR}{P + R}$$

Example: Token-Level F1

- Gold Answer: "*Paris is the capital of France*"
- Generated Answer: "*Paris is capital of France*"
- Common tokens: Paris, is, capital, of, France (5 tokens)
- Compute Precision & Recall
 - Precision = $5 / 5 = 1.0$
 - Recall = $5 / 6 \approx 0.83$

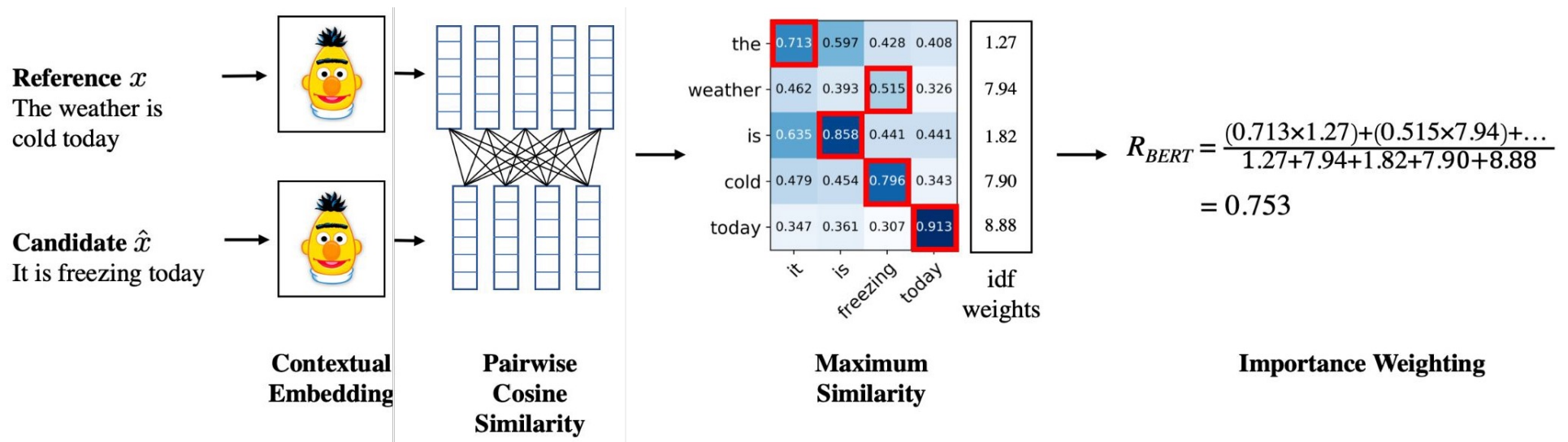
$$F1 = \frac{2PR}{P + R}$$

$$F1 = \frac{2(1.0)(0.83)}{1.83} \approx 0.91$$

End-to-End Answer Quality

- BERTScore

- Uses contextual embeddings to measure semantic similarity
- Captures meaning beyond exact word overlap



Automatic vs Human Evaluation

- Automatic metrics are useful but limited:
 - Token-Level F1: overlap-heavy; weak on factuality
 - BERTScore: semantic similarity; still imperfect
- Human evaluation is often needed for:
 - Correctness
 - Faithfulness (supported by evidence)
 - Completeness
 - Clarity

RAG vs Fine-Tuning

- RAG
 - Uses external retrieval (knowledge can be updated by updating the corpus/index)
 - No weight updates
 - Often higher inference complexity (retrieval + generation)
- Fine-tuning
 - Updates model weights (parametric knowledge)
 - Training can be expensive; updates are less frequent
 - Good for style, format, and domain adaptation (not just “adding facts”)

Building a Good RAG System

- Retrieval Quality
 - Chunking strategy
 - Embedding model
 - Query representation
- Generation Quality
 - Prompt design
 - Context formatting
- System Evaluation
 - Retrieval metrics
 - End-to-end answer quality

Summary

- RAG augments an LLM with external, non-parametric memory (retrieval)
- Pipeline: Retrieve relevant chunks → provide as context → generate answer
- Core components: Retrieval | Embeddings | Prompting | Evaluation
- Retrieval quality bounds answer quality
- Answer correctness $\neq$ evidence-grounded faithfulness